ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios

Do Zero ao MLOps

OpAIoT
Optimization • AI • IoT

Monitoria

Segurança em MQTT: de checksums à autenticação criptográfica

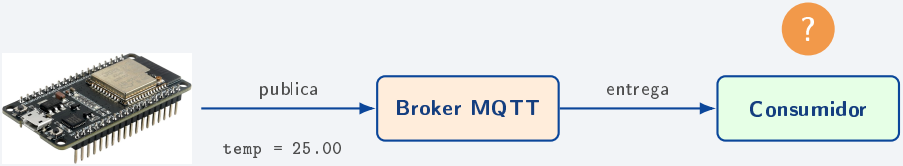
Roteiro

- A pergunta central
- Checksum e CRC
- Funções hash criptográficas
- MAC e HMAC
- Segurança em camadas
- Laboratório ESP32 + MQTT
- Síntese e discussão
- Avaliação da aula

1 A pergunta central

OpAIoT
Optimization • AI • IoT

A pergunta que guia a aula

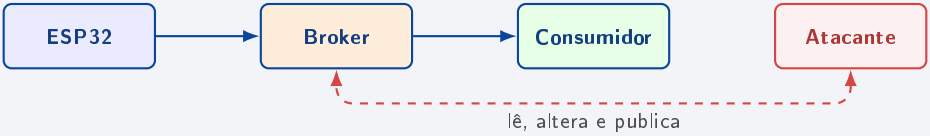


PERGUNTA PARA A TURMA

Como o consumidor pode confiar que a mensagem **não foi adulterada** e que foi **produzida por uma fonte autorizada**?

Cada mecanismo da aula surge como resposta à insuficiência do anterior.

Ameaças em um fluxo MQTT



Ameaça	O atacante...	Propriedade violada
escuta	lê o conteúdo em trânsito	**confidencialidade**
adulteração	altera uma mensagem capturada	**integridade**
falsificação	publica uma leitura inventada	**autenticidade**
replay	reenvia uma mensagem antiga	**atualidade**

Cada ameaça viola uma propriedade diferente.

2 Checksum e CRC

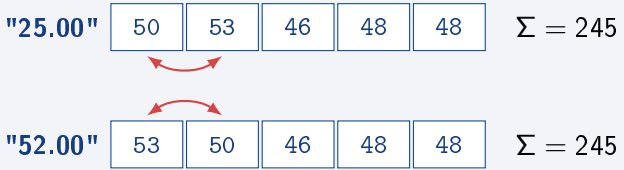
Checksum: somar e reduzir

$$C(m) = \left(\sum_{i=1}^{n} b_i \right) \bmod 256$$



O emissor envia $(m, C(m))$; o receptor recalcula $C(m)$ e compara.
Barato, simples e útil contra **erros acidentais** de transmissão.

Limitação: alterações que se cancelam



Trocar dois dígitos de lugar muda a temperatura de **25** para **52** °C
e o checksum **não percebe**: a soma é insensível à ordem dos bytes.

E um atacante que altera a mensagem também recalcula o checksum.

O que o CRC garante

Com um gerador $G(x)$ de grau r bem escolhido, o CRC detecta:

todos os erros de 1 bit	**todas as rajadas de comprimento $\leq r$**	**todo número ímpar de bits errados**
se G tem ≥ 2 termos	bits vizinhos corrompidos	se $(x+1)$ divide $G(x)$

Está em toda parte: **Ethernet** e **Wi-Fi** (CRC-32), arquivos **ZIP** e **PNG**, barramentos como CAN.

Projetado contra um canal ruidoso, não contra um adversário.

E se o atacante recalcular o CRC?

PERGUNTA PARA A TURMA

Se um atacante modifica a temperatura **e** o CRC, como o receptor identifica a adulteração?

E se o atacante recalcular o CRC?

PERGUNTA PARA A TURMA

> Se um atacante modifica a temperatura **e** o CRC, como o receptor identifica a adulteração?

```
CRC-32("esp32-01|1|25.00")   = 0x52aecc14
CRC-32("esp32-01|1|26.00")   = 0x401b63fa
```

Não identifica. O CRC é público e determinístico: qualquer um o recalcula. Além disso, é linear — dá para prever o efeito de uma alteração no CRC.

Precisamos de uma função impossível de manipular na prática.

3 Funções hash criptográficas

Uma impressão digital de tamanho fixo

$$H : \{0,1\}^* \longrightarrow \{0,1\}^{256}$$



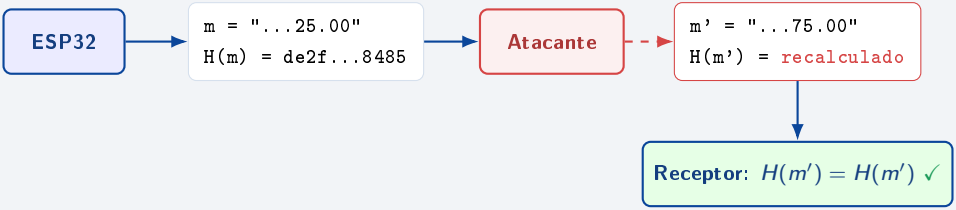
Determinística e rápida de calcular — mas, ao contrário do CRC, **projetada para resistir a um adversário**. Padronizada no NIST FIPS 180-4.

Três propriedades de segurança

Propriedade	Deve ser inviável...	Custo
Resistência à pré-imagem	dado h, achar m com $H(m) = h$	2^{256}
Resistência à 2ª pré-imagem	dado m, achar $m' \neq m$ com $H(m') = H(m)$	2^{256}
Resistência à colisão	achar quaisquer $m \neq m'$ com $H(m) = H(m')$	2^{128}

Para o atacante do slide anterior: achar **outra** temperatura com o **mesmo** resumo é um problema de segunda pré-imagem — fora de alcance.

A limitação: o hash não tem segredo



Qualquer pessoa calcula $H(m')$. O resumo só prova integridade se chegar por um **canal confiável** — e não pelo mesmo canal da mensagem.

Falta um ingrediente que o atacante não tenha: um segredo.

4 MAC e HMAC

OpAIoT
Optimization • AI • IoT

A ideia: acrescentar uma chave secreta

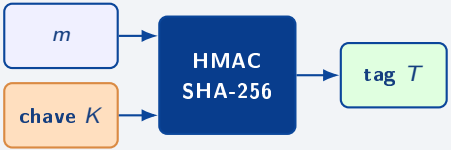
OpAIoT
Optimization • AI • IoT

Hash simples



qualquer um calcula

HMAC



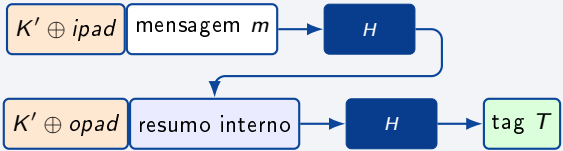
só quem tem K calcula

$$T = \mathrm{HMAC}_K(m)$$

Um hash público pode ser recalculado por qualquer um; uma tag válida exige a chave.

A construção HMAC

$$\mathrm{HMAC}_K(m) = H\Big((K' \oplus opad) \parallel H\big((K' \oplus ipad) \parallel m\big)\Big)$$



1ª passada: mistura chave e mensagem

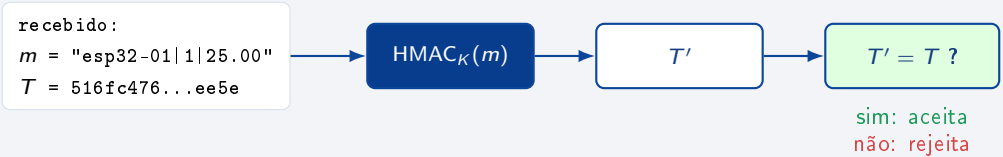
2ª passada: "sela" o resultado

K': chave completada até o bloco de 64 bytes $\parallel$: concatenação

ipad: byte 0x36 repetido *opad*: byte 0x5c repetido

Definido no RFC 2104 e padronizado no NIST FIPS 198-1.

Verificação no receptor



chave didática:	`"chave-secreta-da-aula"`
HMAC_K`("...|1|25.00")`	`516fc47614520e71726f4aac9e8d6822...`
HMAC_K`("...|1|26.00")`	`543203aa8289430d7526a812ae5e9d9b...`

Compare em **tempo constante** (`hmac.compare_digest`): comparar byte a byte e parar no primeiro erro vaza, pelo tempo, quantos bytes da tag estavam certos.

O que o HMAC não é

Não é criptografia

O payload continua legível por quem o intercepta.

Não é assinatura digital

Quem verifica também pode gerar a tag: não há não repúdio.

Não garante atualidade

Uma cópia exata de uma mensagem válida continua válida.

É comum dizer que o dispositivo "assina" a mensagem — mas o HMAC é **autenticação simétrica**: o mesmo segredo K gera e verifica.

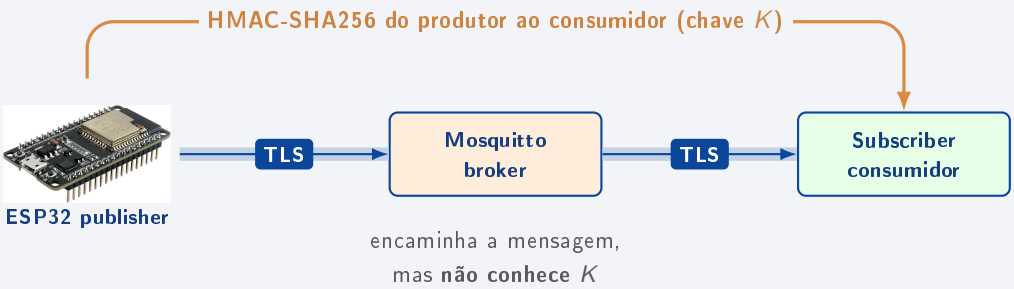
HMAC entrega integridade e autenticidade. Sigilo e atualidade vêm de outros mecanismos.

5

Segurança no MQTT em camadas

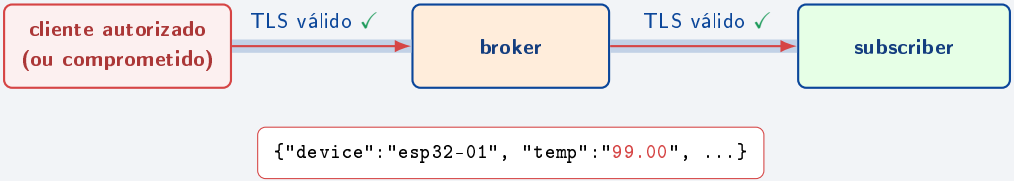
OpAIoT
Optimization • AI • IoT

Transporte × aplicação



O TLS protege **cada salto** (cliente ↔ broker). O HMAC protege o **conteúdo** de ponta a ponta, atravessando o broker sem que ele precise do segredo.

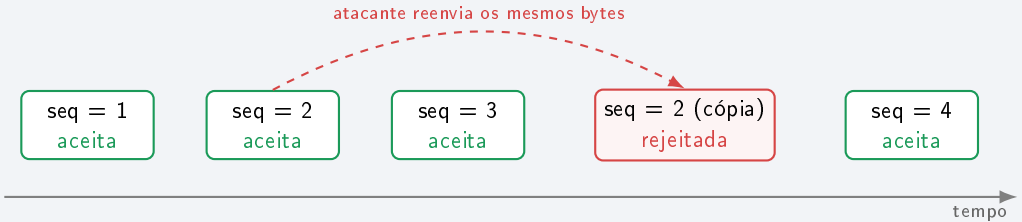
Por que o TLS não basta?



O TLS garante que a conversa com o broker é privada e íntegra — **não** que o conteúdo publicado seja verdadeiro. O broker decifra, roteia e cifra de novo.

Um canal seguro pode transportar, com perfeição, uma mensagem maliciosa.

Replay: o HMAC continua válido

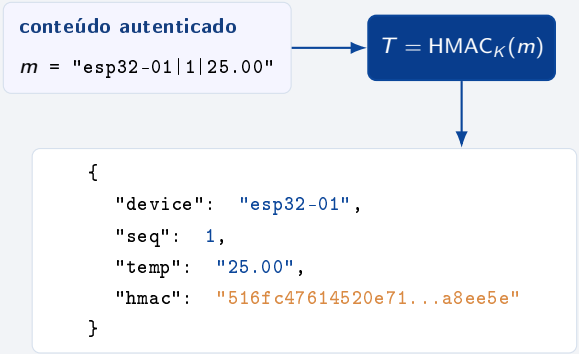


HMAC ✓, mas $2 \leq 3$

$$\text{aceitar } (d, s, T) \iff \text{HMAC}_K(m) = T \ \wedge \ s > s_{\text{último}}(d)$$

Ao aceitar, atualiza-se $s_{\text{último}}(d) \leftarrow s$, guardado por dispositivo d.

A mensagem do laboratório

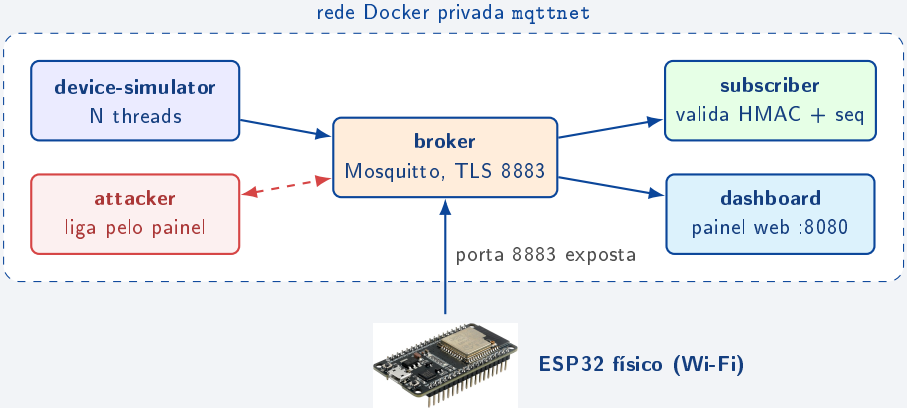


O seq está **dentro** do conteúdo autenticado: o atacante não pode trocá-lo para driblar a regra de sequência sem invalidar a tag.

6 Laboratório ESP32 + MQTT

OpAIoT
Optimization • AI • IoT

Arquitetura do laboratório



Preparação do ambiente

Na máquina que executará o Docker:

```
git clone https://github.com/JunioCesarFerreira/esp32-mqtt-security-lab
cd esp32-mqtt-security-lab

hostname -I                                          # descubra o IP do host
./setup.sh 192.168.1.50                              # substitua pelo IP real

docker compose up -d --build
docker compose logs -f subscriber
```

Abra o painel em `http://localhost:8080`. Se a porta estiver ocupada, ajuste `DASHBOARD_PORT` no `.env` da raiz.

O simulador dispensa o ESP32 físico. Com a placa, configure o Wi-Fi em `esp32/secrets.h` e grave `esp32/esp32.ino` (biblioteca `PubSubClient`).

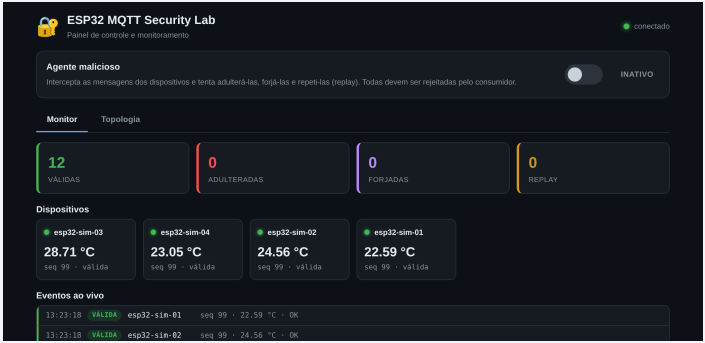
O que o `setup.sh` gera

- CA própria e certificado TLS do broker para o IP informado;
- um usuário MQTT por serviço, com ACL por tópico;
- uma chave HMAC aleatória de 32 bytes.

O **broker** e o **attacker** recebem credenciais MQTT válidas, mas **nunca** a chave K.

ESP32	✓ conhece K
device-simulator	✓ conhece K
subscriber	✓ conhece K
dashboard	✓ conhece K
broker	× sem K
attacker	× sem K

O painel de controle



botão do atacante · contadores por veredito · cartão por dispositivo

Atacante desligado: só telemetria válida.

Experimento 1 — o ESP32 autentica

Trecho de `esp32/esp32.ino` (HMAC com `mbedtls`, já incluso no core do ESP32):

```
String m = String(DEVICE_ID) + "|" + String(seqNumber) + "|" + tempStr;   // "esp32-01|1|25.00"

mbedtls_md_setup(&ctx, mbedtls_md_info_from_type(MBEDTLS_MD_SHA256), 1);   // 1 = modo HMAC
mbedtls_md_hmac_starts(&ctx, key, keyLen);
mbedtls_md_hmac_update(&ctx, (const unsigned char *)m.c_str(), m.length());
mbedtls_md_hmac_finish(&ctx, digest);                                       // 32 bytes -> 64 hex

snprintf(payload, sizeof(payload),
         "{\"device\":\"%s\",\"seq\":%lu,\"temp\":\"%s\",\"hmac\":\"%s\"}",
         DEVICE_ID, (unsigned long)seqNumber, tempStr, hmacHexStr);
mqttClient.publish(MQTT_TOPIC, payload);
```

O `seqNumber` cresce a cada publicação; a chave vem de `secrets.h`, gerado pelo `setup.sh`.

Experimento 1 — o subscriber verifica

Trecho simplificado de subscriber/subscriber.py:

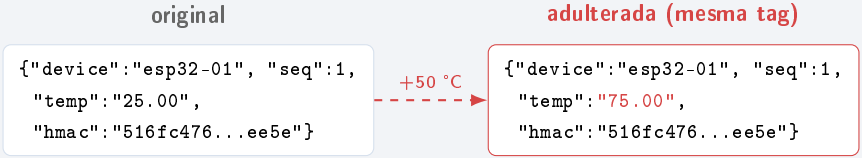
```
def sign(device, seq, temp):
    m = f"{device}|{seq}|{temp}".encode()
    return hmac.new(HMAC_KEY, m, hashlib.sha256).hexdigest()

if not hmac.compare_digest(data["hmac"], sign(device, seq, temp)):
    rejeitar("HMAC invalido")                    # adulterada ou forjada
elif device in last_seq and seq <= last_seq[device]:
    rejeitar("replay ou fora de ordem")          # copia de mensagem antiga
else:
    last_seq[device] = seq
    aceitar()
```

```
[VALIDA] device=esp32-01 seq=1
temperatura=25.00 C
```

Experimento 2 — adulteração

Ligue o atacante no painel. Ele intercepta cada leitura e a republica alterada:

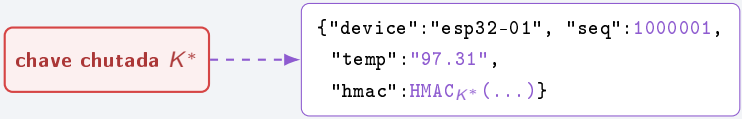


$$\mathrm{HMAC}_K(\texttt{"esp32-01|1|75.00"}) \neq \texttt{516fc476...ee5e}$$

ADULTERADA rejeitada: *HMAC não confere*

Experimento 3 — falsificação

O atacante cria uma leitura nova e a assina com uma chave **própria**, aleatória:



$$\Pr\left[\text{acertar uma tag válida sem } K\right] \approx 2^{-256}$$

FORJADA rejeitada: *assinatura inválida*

Publicar no broker não é o mesmo que produzir dados aceitos.

Experimento 4 — replay

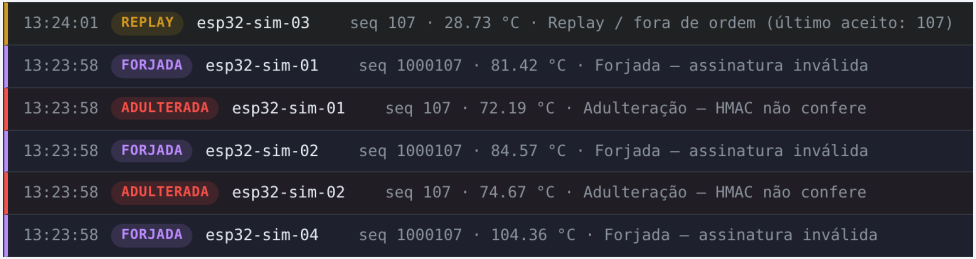
Três segundos depois, o atacante reenvia a mensagem original, **byte a byte**:



REPLAY rejeitada: *replay / fora de ordem* — pela regra de sequência, não pelo HMAC

É o único dos três ataques que o HMAC sozinho deixaria passar.

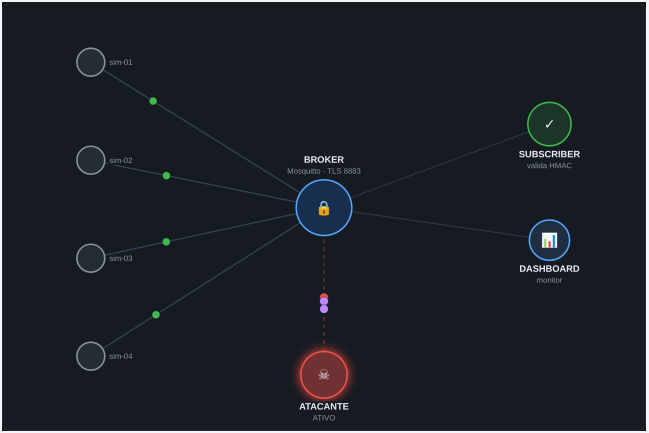
O que observar no painel



Para cada leitura legítima, o atacante gera uma **adulterada**, uma **forjada** e, 3 s depois, um **replay**.

Todas são rejeitadas — e cada uma pelo motivo certo.

Topologia ao vivo



verde: telemetria dos dispositivos

vermelho, **roxo**, **laranja**: mensagens injetadas pelo atacante

o broker entrega tudo; quem decide é o subscriber

Exercício: a sequência é um estado

ESP32 (RAM)

```
uint32_t seqNumber = 0;
```

subscriber (memória)

```
last_seq_by_device = {}
```

Evento	Consequência	Risco
ESP32 reinicia	seq volta a 1 e parece antigo	leituras legítimas rejeitadas
subscriber reinicia	esquece o último seq	janela para replay
chave única	um dispositivo forja outro	comprometimento amplo

PERGUNTA PARA A TURMA

> Como corrigir cada caso sem perder a proteção contra replay?

Caminhos de solução

Persistir o estado
seq na flash do ESP32 (NVS) e no armazenamento do subscriber.

Épocas de inicialização
contador de boots e junto do seq:

$$(e, s) > (e_{\text{últ}}, s_{\text{últ}})$$

comparação lexicográfica: primeiro e, depois s

Tempo ou desafio
timestamp com janela de aceitação, ou *nonce* enviado pelo consumidor (desafio–resposta).

Uma chave por dispositivo
derivada de uma chave mestra, com rotação:

$$K_d = \text{HMAC}_{K_{\text{mestra}}}(d)$$

Segurança não é só algoritmo: é também estado e hipóteses de confiança.

7 Síntese e discussão

Quadro de síntese

	Detecta corrupção	Autentica conteúdo	Confidencialidade	Protege contra replay
Checksum	~ limitada	✗	✗	✗
CRC	✓ acidental	✗	✗	✗
SHA-256 isolado	✓ c/ ref. confiável	✗	✗	✗
HMAC-SHA256	✓	✓ chave compart.	✗	✗
HMAC + sequência	✓	✓	✗	✓ com estado
TLS	✓ por conexão	~ os pares	✓	~ só na conexão

Nenhuma linha tem tudo: a segurança efetiva nasce da **composição** das camadas.

Roteiro

Fechamento

- **Checksum e CRC:** erros acidentais.
- **SHA-256:** resiste a adversários, sem segredo.
- **HMAC:** integridade e autenticidade.
- **Sequência:** atualidade — exige gerir estado.
- **TLS:** protege o canal, não o conteúdo.



defesa em camadas

A segurança efetiva nasce da composição das camadas.

Referências

- H. Krawczyk, M. Bellare, R. Canetti. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, IETF, 1997.
- NIST. *The Keyed-Hash Message Authentication Code (HMAC)*. FIPS 198-1, 2008.
- NIST. *Secure Hash Standard (SHS)*. FIPS 180-4, 2015.
- OASIS. *MQTT Version 5.0*. OASIS Standard, 2019 — seção 5, *Security*.
- W. W. Peterson, D. T. Brown. *Cyclic Codes for Error Detection*. Proceedings of the IRE, 1961.
- Laboratório da aula: https://github.com/JunioCesarFerreira/esp32-mqtt-security-lab



Obrigado!



opaiot@icmc.usp.br

+55 (16) 99962-7637

PROFESSOR

OpAIoT
Optimization • AI • IoT